

09. One-to-Few & One-to-Many Relationships

✦ Relational SQL vs NoSQL Data Modeling

SQL me Foreign Keys aur JOINS use hote hain. MongoDB me hamare paas 2 primary techniques hain: **Embedding (Denormalized)** aur **Referencing (Normalized)**.

Approach 1: One-to-Few (Embedded Documents)

Jab child items ki sankhya kam aur fixed ho (e.g. User ke 2-3 addresses). Child documents seedha parent ke array me store hote hain:

```
const userSchema = new mongoose.Schema({
  username: String,
  addresses: [
    { _id: false, location: String, city: String } // Embedded directly!
  ]
});
```

One-to-Few Schema – Embedding

Approach 2: One-to-Many (Child Referencing)

Jab child items zyada ho sakte hain (e.g. Customer with multiple Orders). Parent document me child ke **ObjectId** ka array store hota hai:

```
// 1. Order Schema
const orderSchema = new mongoose.Schema({ item: String, price: Number });
const Order = mongoose.model("Order", orderSchema);

// 2. Customer Schema (Storing Array of ObjectIds)
const customerSchema = new mongoose.Schema({
  name: String,
  orders: [{ type: mongoose.Schema.Types.ObjectId, ref: "Order" }]
});
const Customer = mongoose.model("Customer", customerSchema);
```

One-to-Many Schema – Referencing

✦ The Power of populate()

.populate("orders") un ObjectIds ko actual Order documents se replace kar deta hai:

```
const customer = await Customer.findOne({ name: "Rahul" }).populate("orders");
console.log(customer);
```

Query with populate()



One-to-Squillions & Architecture Summary



Approach 3: One-to-Squillions (Parent Referencing)

Kyu chahiye? MongoDB me ek single document ka maximum size **16 MB** ho sakta hai. Agar ek Post par 10 lakh comments hon, toh parent document ke andar ObjectIds ka array 16 MB limit ko exceed kar dega!



16 MB DOCUMENT LIMIT: Never store unbounded arrays (thousands/millions of items) inside a single document! Always use Parent Referencing!

Solution: Child document ke andar Parent ka reference store karo:



Parent Referencing Schema

```
// 1. Post Schema (No comments array!)
const postSchema = new mongoose.Schema({ title: String, content: String });

// 2. Comment Schema (Stores Parent Reference)
const commentSchema = new mongoose.Schema({
  text: String,
  post: { type: mongoose.Schema.Types.ObjectId, ref: "Post" } // 🐵 Parent reference
});
```



Master Relationship Decision Matrix

Relationship Type	Data Size	Schema Design	Real-world Use Case
One-to-Few	Few items (< 10)	Embedded (inside parent)	User addresses, phone numbers
One-to-Many	Medium (10 to ~1000)	Child Referencing (Array of IDs in parent)	Customer orders, shopping cart items
One-to-Squillions	Huge (1,000 to Millions)	Parent Referencing (Parent ID in child)	Post comments, system logs, followers



Topper's Quick Revision Checklist

- ✓ **mongosh:** use `db.col.insertOne()`, `db.col.find()`, `db.col.updateOne()`.
- ✓ **BSON:** Binary JSON with rich data types (Date, ObjectId) & fast traversal.
- ✓ **Mongoose Model:** "User" automatically creates "users" collection.
- ✓ **Updates:** Always use `{ runValidators: true }` on `findByIdAndUpdate!`
- ✓ **wrapAsync:** `fn(req, res, next).catch(next)` eliminates ugly try-catch blocks.
- ✓ **Populate:** Replaces ObjectIds with actual linked documents in memory.